ASSIGNMENT - 1

NAME: M. Rumana khan

REG NO: 192471025

COURSE CODE / NAME: CSA0610 - Design and Analysis Of Algorithms

---

Assignment Title: Design, Analyse and Validate an Efficient Search Strategy for a Large-Scale Student Record System

---

Introduction

A university student record system must be able to find a student's details quickly using their unique Registration Number. When the number of records increases from 1,000 to more than 1,000,000, the search algorithm becomes an important factor in the system's performance.

The aim of this assignment is to study different searching algorithms, analyse their time complexity mathematically, implement them, and compare their performance on datasets of different sizes. The final objective is to choose an algorithm that provides efficient and reliable searching while considering preprocessing costs and future growth.

Task 1 – Design the Algorithms

I treat each student record as a key (the registration number), and the job is to decide whether a given key exists and to retrieve its record. I picked three strategies that are different in kind, not just in code style: one that ignores structure, one that uses ordering, and one that uses direct addressing.

Approach A – Linear Search (iterative, brute force)

Idea: start at the first record and check each one in turn until the key is found or the records run out.

Preprocessing: none. The data can stay in whatever order it arrives.

Why I chose it: it is the baseline. Any smarter method has to beat it, and it is the only option for unsorted data that is searched rarely.

```
LinearSearch(A[0..n-1], key)
  for i ← 0 to n-1
    if A[i] = key
      return i
  return -1
```

Approach B – Binary Search (recursive, decrease-and-conquer)

Idea: keep the records sorted. Compare the key with the middle record. If the key is smaller, only the left half can contain it; if larger, only the right half. Every step throws away half of the remaining records.

Preprocessing: sort the array once, Θ(n log n).

Why I chose it: it takes advantage of order in the data, which linear search ignores completely.

```
BinarySearch(A, key, low, high)
  if low > high
    return -1                  // base case: empty range
  mid ← ⌊(low + high) / 2⌋
  if A[mid] = key
```

```
    return mid
  else if key < A[mid]
    return BinarySearch(A, key, low, mid-1)
  else
    return BinarySearch(A, key, mid+1, high)
```

Approach C – Hash Table Search (iterative, transform-and-conquer)

Idea: skip comparing positions altogether. A hash function h(key) = key mod m tells us which bucket the record belongs in, and we look only inside that bucket. Collisions are handled by chaining (each bucket holds a short list).

Preprocessing: build the table by inserting all n keys, expected Θ(n). I use a table size m ≈ n / 0.75, so the load factor α stays near 0.75.

Why I chose it: it is a third idea entirely. Instead of narrowing down to the answer, it jumps straight to it.

```
HashSearch(T[0..m-1], key)
  i ← key mod m
  for each record r in the list T[i]
    if r = key
      return r
  return NOT_FOUND
```

Approach	Design strategy	Style	Preprocessing
A. Linear search	Brute force	Iterative	None
B. Binary search	Decrease-and-conquer	Recursive	Sort, Θ(n log n)
C. Hash table	Transform-and-conquer (hashing)	Iterative	Build table, Θ(n) expected

In short, three genuinely different strategies, with binary search as the recursive one and the other two iterative.

Task 2 – Basic Operation

For all three algorithms the basic operation is a key comparison, but it sits in a different place in each one.

Algorithm	Basic operation	Where it happens
Linear search	Comparison A[i] = key	Inside the loop body, once for every record visited
Binary search	Comparison with A[mid] (equal / smaller / larger, counted as one three-way comparison)	Once per recursive call
Hash search	Comparison r = key	Inside the bucket scan, once for every record in the chain that gets examined

Why is comparison the right thing to count? In each algorithm it is the step executed most often. Everything else (incrementing i, computing mid, computing key mod m) happens at most once per comparison, so it only changes the

constant factor and never the growth rate. Comparing keys is also the only step that actually touches the data, so it reflects the real work being done.

How the count changes as n grows.

- Linear search: the count grows in direct proportion to n, since in the worst case every record is checked.
- Binary search: the range halves every time, so the count grows with the number of times n can be halved, which is $\log_2 n$.
- Hash search: the count depends on the chain length, which depends on the load factor $\alpha = n/m$. Because I keep α roughly constant as the table grows, the count does not grow with n at all.

In short, counting key comparisons works for all three, and how that count reacts to n is exactly what separates them.

Task 3 – Time-Complexity Function

The input size n is the number of student records. C(n) is the number of key comparisons.

A. Linear search

A successful search for the record in position i takes i comparisons. If every position is equally likely, the average is:

$$\frac{1 + 2 + \dots + n}{n} = \frac{n(n+1)/2}{n} = \frac{n+1}{2}$$

An unsuccessful search checks every record, so it takes exactly n comparisons. If a fraction p of searches succeed, the average is:

$$C_{avg}(n) = p \cdot \frac{n+1}{2} + (1-p) \cdot n$$

Preprocessing is 0, so the cost of q searches is $q \cdot n$.

B. Binary search

At step k the search looks at one element of a range of size at most $n / 2^{(k-1)}$. The search must stop once the range is empty, so the number of steps L satisfies $n / 2^L < 1$, which gives:

$$C_{worst}(n) = \lfloor \log_2 n \rfloor + 1$$

For the average successful case, picture the search as a tree. One element is examined at step 1, two at step 2, four at step 3, and in general 2^{k-1} elements at step k. So:

$$C_{avg} = \frac{1}{n} \sum_{k=1}^{\log_2 n} k \cdot 2^{k-1} \approx \log_2 n - 1$$

Preprocessing: sorting n records with a comparison sort costs Θ(n log n). The cost of q searches is n log n + q log n = (n + q) log n.

C. Hash search

With n keys spread over m buckets and a reasonably uniform hash function, the average chain length is α = n/m.

- Successful search: about 1 + α/2 records scanned.
- Unsuccessful search: the whole chain, about α records.

With α ≤ 0.75 both are a small constant, so C_avg(n) = Θ(1). In the worst case every key lands in one bucket and the search scans n records.

Preprocessing: n insertions at Θ(1) expected each, so Θ(n). The cost of q searches is n + q(1 + α/2) = Θ(n + q).

Preprocessing and searching combined (q searches)

Algorithm	**Preprocessing**	**q searches (average)**	**Total**
Linear	**0**	**q·n**	**Θ(q·n)**
Binary	**n log n**	**q log n**	**Θ((n + q) log n)**
Hash	**n**	**q**	**Θ(n + q)**

In short, linear search pays on every single search, binary search pays a big one-time sorting fee and then very little per search, and hashing pays a smaller one-time fee and almost nothing per search.

Task 4 – Best, Average and Worst Case

Linear search

- Best case: the key is the very first record. One comparison, $\Theta(1)$.
- Average case: the key is equally likely to be anywhere, giving $(n+1)/2$ comparisons. If misses are common the average moves toward n, as the formula in Task 3 shows.
- Worst case: the key is the last record or is not in the database. n comparisons. This matters in practice, because a mistyped registration number is not unusual.
- Why: the loop has no way to skip anything, so the position of the key decides the cost entirely.

Binary search

- Best case: the key is exactly the middle element on the first look. One comparison.
- Average case: about $\log_2 n - 1$ comparisons for a key that exists. Half of all records sit at the deepest level of the implicit search tree, so a typical hit is found late.
- Worst case: the key is missing or sits at the deepest level, giving $\lfloor \log_2 n \rfloor + 1$ comparisons. For n = 1,000,000 that is 20.
- Why: the halving happens no matter where the key is, so the gap between best and worst case is only about log n and the cost cannot blow up.

Hash search

- Best case: the bucket is empty (an unsuccessful search with zero comparisons) or the key is first in its chain (one comparison).
- Average case: chains average $\alpha \approx 0.75$ long, giving about $1 + \alpha/2 \approx 1.4$ comparisons for a hit.
- Worst case: every registration number hashes to the same bucket, so one chain holds all n entries and the search degrades to $\Theta(n)$.

- Why: hashing depends on keys spreading evenly. Registration numbers are often sequential or share a prefix, so a poor hash function (for example one that looks only at the leading digits) could cause this. Using key mod m with an odd table size avoids it for typical numbering schemes.

In short, linear search has the widest gap between best and worst case, binary search has a narrow gap guaranteed by the halving, and hashing is nearly always at the good end but has a pathological worst case.

Task 5 – Asymptotic Analysis

Algorithm	Best case	Worst case	Average case
Linear	Ω(1)	O(n)	Θ(n)
Binary	Ω(1)	O(log n)	Θ(log n)
Hash	Ω(1)	O(n)	Θ(1) with constant α

Why Θ for the averages. The average counts are bounded above and below by the same order (for example (n+1)/2 is both O(n) and Ω(n)), so Θ is the honest notation. For an algorithm as a whole, the best and worst cases differ, so only Ω and O apply to the algorithm overall.

Behaviour for large n. The order of growth is n, then log n, then constant. When the dataset grows 1,000-fold, from 1,000 to 1,000,000 records:

- Linear search needs about 1,000 times as many comparisons.
- Binary search goes from about 10 to about 20 comparisons, just double.
- Hash search stays at about one comparison.

What this means for the university. At 1,000 records the three are practically indistinguishable and any of them would do. At 1,000,000+ records linear search becomes a real cost, while the other two are still essentially instant. A tenfold increase in the database adds only about 3 comparisons to binary search and nothing to the hash table.

In short, the notation matters less than what it predicts, which is that only linear search gets noticeably worse as the university grows.

Task 6 – Recurrence Relation (Binary Search)

I formulate the recurrence directly from the recursive code in Task 1.

- Input size of the original problem: n = high − low + 1, the number of records still under consideration.
- Size of the subproblem: after comparing with A[mid], the function recurses on either the left part or the right part, never both. Each part has at most ⌊n/2⌋ records.
- Number of recursive calls: exactly one, because only one of the two branches is taken.
- Work outside the recursive call: computing mid and doing the comparison, a constant amount c.
- Base case: an empty range (n = 0) returns immediately in constant time. A range of one element also costs a constant (one comparison, then one call on an empty range).

Putting these together:

$$T(n) = T(\lfloor n/2 \rfloor) + c, \quad T(0) = T(1) = c$$

For the analysis I assume n = 2^k so the floor disappears, and the worst case where the search always continues into the larger half.

Task 7 – Solving the Recurrence

I use the iteration (backward substitution) method to show the steps, then confirm the answer with the Master Theorem. I chose iteration because the recurrence has a single recursive term, so unrolling it is easy and the pattern appears quickly.

Let n = 2^k. Unrolling:

$$\begin{aligned}$$

$$T(n) &= T(n/2) + c \\$$

$$&= T(n/4) + 2c \\$$

&= T(n/8) + 3c \\

&\;\;\vdots \\

&= T(n/2^i) + i \cdot c

\end{aligned}

The recursion stops when n / 2^i = 1, which means i = $\log_2 n$. Substituting:

T(n) = T(1) + c \log_2 n = c + c \log_2 n \;\Rightarrow\; T(n) = \Theta(\log n)

Check with the Master Theorem. The recurrence fits T(n) = aT(n/b) + f(n) with a = 1, b = 2 and f(n) = c = Θ(1). Then n^(log_b a) = n^($\log_2$ 1) = n^0 = 1. Since f(n) = Θ(n^0), this is Case 2, which gives T(n) = Θ(n^0 · log n) = Θ(log n). Both methods agree.

What it means. The cost depends only on how many times n can be cut in half, so doubling the database adds just one extra step. Even for 1,000,000 records the recursion is only about 20 calls deep, so there is also no risk of running out of stack space.

In short, T(n) = Θ(log n), confirmed two different ways.

Task 8 – Implementation and Testing

I implemented all three algorithms in Python (full code in the appendix), following the pseudocode from Task 1 line by line. Each version also returns its comparison count, so the measurements connect directly to the analysis.

Procedure

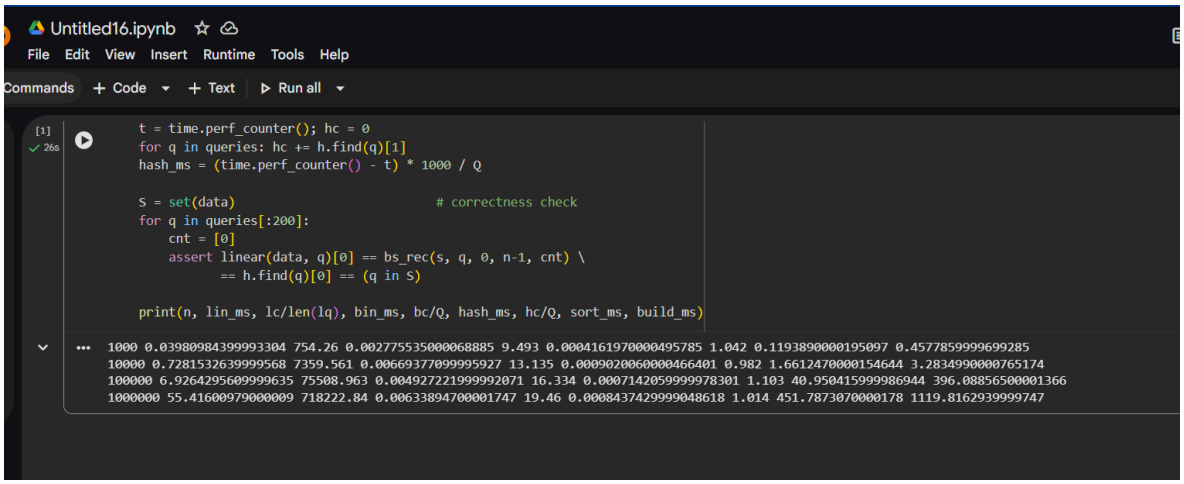
1. Generate n unique 8-digit registration numbers at random, with a fixed seed (42) so the run can be repeated, for n = 1,000; 10,000; 100,000 and 1,000,000.
2. Use the same list of records for all three algorithms. Linear search uses it unsorted, binary search uses a sorted copy, and the hash table is built from the same list.

3. Build 1,000 test queries per size: 500 keys that exist and 500 that do not, shuffled together, so both successful and unsuccessful searches are covered.
4. Time each algorithm over the same queries and report the average per query. Preprocessing (sorting, building the hash table) is timed separately.
5. For n = 1,000,000, linear search was run on 100 queries instead of 1,000, purely to save time, and averaged.

Correctness check. For 200 queries at every size, an assert confirms that all three algorithms agree with each other and with Python's own set membership test. No assertion failed.

Test cases covered: key present, key absent, and the worst case for linear search (an absent key, which forces a full scan).

In short, same data, same queries, same machine, preprocessing timed separately, and all three algorithms agree on every answer.



Task 9 – Experimental Performance

The tables below give the measured results. Each row is an average over the same 1,000 queries (half hits, half misses), except linear search at n = 1,000,000, which used 100 queries.

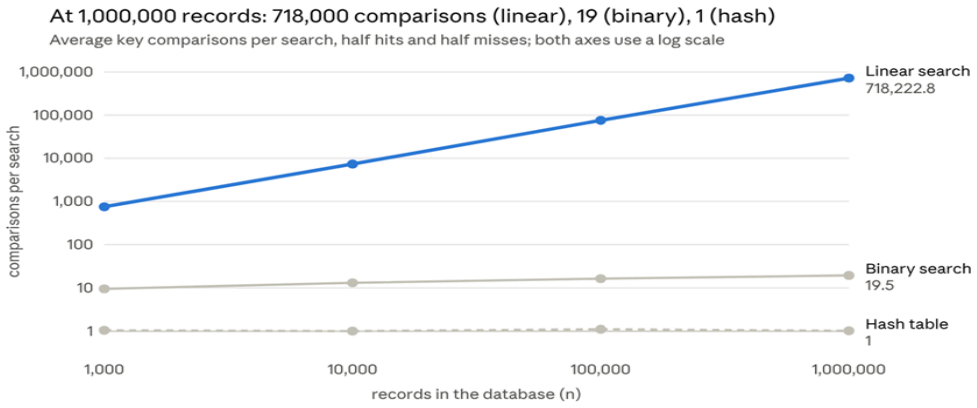
Table 1 – Average cost per search

n	Linear: comparisons	Linear: time (ms)	Binary: comparisons	Binary: time (ms)	Hash: comparisons	Hash: time (ms)
1,000	754.3	0.0398	9.49	0.0028	1.04	0.00042
10,000	7,359.6	0.7282	13.14	0.0067	0.98	0.00090
100,000	75,509.0	6.9264	16.33	0.0049	1.10	0.00071
1,000,000	718,222.8	55.4160	19.46	0.0063	1.01	0.00084

Table 2 – One-time preprocessing cost

n	Linear search	Sorting for binary search (ms)	Building the hash table (ms)
1,000	none	0.12	0.46
10,000	none	1.66	3.28
100,000	none	40.95	396.09
1,000,000	none	451.79	1,119.82

Graph 1 – Comparisons per search against n



On log axes, linear search is a steep straight line (comparisons grow 1,000 times when n does), binary search rises only slowly, and the hash table stays flat at about one. The time columns in Table 1 follow the same shapes.

In short, linear search grows roughly in step with n, binary search barely moves, and hashing stays flat.

Task 10 – Theory vs Experiment

Linear search matches the theory closely. With half the queries being hits (average n/2) and half being misses (exactly n), the predicted average is 0.5·(n/2) + 0.5·n = 0.75n. The measured counts, 754, 7,360, 75,509 and 718,223, work out to 0.754n, 0.736n, 0.755n and 0.718n. The last one is a little lower because at n = 1,000,000 I only ran 100 queries, so the hit and miss mix was not exactly half and half. Time grew about 1,400 times from n = 1,000 to 1,000,000, a bit more than the 1,000 times the comparison count suggests, probably because the larger array does not fit in the processor's fast cache. I did not test that directly.

Binary search matches the $\log_2 n$ prediction. The expected count is about 9 to 10 at n = 1,000 and 19 to 20 at n = 1,000,000. The measured values are 9.49 and 19.46, so comparisons only doubled while n grew 1,000-fold. Time grew about 2.3 times, close to the 2 times seen in comparisons. The middle sizes are a bit irregular (0.0067 ms at 10,000 but 0.0049 ms at 100,000) because each search takes only a few millionths of a second, so small timing noise shows up. I did not investigate further.

Hash search is flat, as predicted. Comparisons stay at about 1 at every size, which fits a load factor held at 0.75. Time rose from 0.00042 ms to 0.00084 ms, about double, while n grew 1,000 times. That is nowhere near linear growth, and it is most likely cache effects on the larger table plus timing noise.

Preprocessing. Sorting grew about 11 times when n grew 10 times (100,000 to 1,000,000), which fits n log n (the theory predicts about 12). The hash build was erratic: 3.3 ms, 396 ms and 1,120 ms at 10,000, 100,000 and 1,000,000 records, where Θ(n) predicts a steady 10-fold rise at each step. The 100,000 figure in particular looks too high. I believe this is a Python effect rather than a problem with the algorithm: building a million small lists puts heavy load on memory allocation and the garbage collector, and runs vary. I did not confirm this. Also, my hash build is a Python loop while sorting uses Python's built-in sort, which is written in C, so the build looks slower here than it would in a fair comparison. In C or Java I would expect it to be close to linear.

Which scales best? The hash table, then binary search, with linear search far behind. One caveat is that all timings come from interpreted Python, so absolute numbers would be lower in a compiled language, but the growth shapes would be the same.

In short, the comparison counts match the theory almost exactly, and the only surprises were in timing, where memory effects and Python overhead show up.

Task 11 – Final Design Recommendation

For the original problem (general lookups on a growing database), I recommend binary search on a sorted array.

Why it beats the alternatives

- Against linear search: at 1,000,000 records, linear search needs about 718,000 comparisons (55.4 ms) per query, against about 19 (0.0063 ms) for binary search. The 452 ms sorting cost is repaid after roughly 8 searches (452 ms divided by about 55.4 ms saved per search).
- Against hashing: hashing is faster per search, but in my tests it cost more to build (1.1 s against 0.45 s at 1,000,000 records), it uses extra memory, it has no guaranteed worst case, and it cannot answer ordered questions

such as all students with registration numbers between X and Y. Binary search has a hard guarantee of $\lfloor \log_2 n \rfloor + 1$ comparisons.

Efficiency. Theory: Θ(log n) per search and Θ(n log n) preprocessing. Observed: 19.46 comparisons and 0.0063 ms per search, and a 452 ms sort, at n = 1,000,000.

Trade-offs and limitations

- The array must stay sorted. Inserting one record costs Θ(n) to place it correctly, or a full re-sort. That is fine if records arrive in occasional batches, but not if they arrive constantly.
- Binary search needs random access to the array, so it does not work on a linked list.

Does it stay practical as the database grows? Yes. Going from 1,000,000 to 10,000,000 records adds only about three more comparisons per search. The sort grows slightly faster than linearly but is still a one-time cost.

In short, binary search gives a guaranteed, tiny per-search cost for a modest one-time sort, and it keeps ordered queries available.

Task 12 – Day-4 Design Challenge

Changed requirement: 50,000 searches per hour on 1,000,000 records, with new records added only once a day.

A. Impact Analysis

- What changed: the original problem never said how often searches happen. Now it does, and it is high: about 50,000 per hour, roughly 14 per second, or about 1.2 million per day. Updates, on the other hand, are rare.
- Assumptions affected: earlier I treated one-time preprocessing as a cost that had to be justified. Now it is paid once a day and spread across over a million searches.
- Why search frequency matters: total cost is preprocessing plus (number of searches × cost per search). With this many searches the per-search cost dominates the total, so small differences per search add up, and per-search cost is exactly where the algorithms differ most.

- Why rare updates matter: the main weakness of a sorted array (expensive inserts) and of a hash table (rebuilding) hardly shows if data changes once a day. The whole index can simply be rebuilt overnight.

B. Evaluation of the Original Design (Binary Search)

Using my measured numbers at n = 1,000,000:

Approach	Cost of 50,000 searches (one hour)	Cost per day (1.2M searches + one rebuild)
Linear search	50,000 × 55.4 ms ≈ 2,771 s (about 46 minutes)	about 18.5 hours of CPU time
Binary search	50,000 × 0.0063 ms ≈ 0.32 s	about 7.6 s + 0.45 s sort ≈ 8.1 s
Hash table	50,000 × 0.00084 ms ≈ 0.04 s	about 1.0 s + 1.1 s build ≈ 2.1 s

- Binary search handles the load without difficulty. The hourly search cost is a fraction of a second, and the sort is a once-a-day cost of under half a second.
- Its preprocessing cost is acceptable. It is paid once a day and is about as large as 8 linear searches.
- Hashing could give a modest further improvement. Honestly, the gap is small in absolute terms, since both finish a whole day's workload in under ten seconds. The hash table's advantage is a lower cost per search that does not grow with n.

C. Adaptation or Redesign

Decision: modify. Keep the sorted array as a backup, but make a hash table the main lookup path.

Revised strategy:

1. Once a day, after the new records arrive, merge them into the sorted array (sorting just the new batch and merging costs $\Theta(n + k)$, cheaper than a full re-sort) and rebuild the hash table in $\Theta(n)$.
2. During the day both structures are read-only, so no locking or insertion logic is needed.

3. Registration-number lookups go to the hash table at Θ(1) expected. Range queries and anything needing order use binary search on the sorted array.

Effect on complexity:

- Search: Θ(log n) becomes Θ(1) expected.
- Daily preprocessing: Θ(n log n) becomes Θ(n) for the hash build, plus Θ(n + k) for the merge.
- Total cost of q searches per day: Θ((n + q) log n) becomes Θ(n + q).
- Memory roughly doubles, since both structures are kept.

I would also keep the load factor at 0.75 or lower and use key mod m with an odd m, to avoid the clustering worst case from Task 4.

D. Evidence-Based Defence

- Search frequency: with 1.2 million searches a day, any cost saved per search is multiplied 1.2 million times. Linear search is clearly out, since it would use about 18.5 hours of CPU time a day on lookups alone, against a few seconds for the other two.
- Preprocessing cost: building the hash table (1.1 s) costs more than sorting (0.45 s) in my tests, but both are paid once a day. The extra cost is recovered within about 20 linear-search equivalents (1,120 ms divided by 55.4 ms), which at this traffic takes seconds.
- Dataset size: binary search needs about 20 comparisons and the hash table about 1, and that lead holds however much the database grows, while the cost of linear search keeps climbing.
- Trade-offs: more memory, no ordered queries from the hash table alone (hence keeping the sorted array), and a theoretical worst case of Θ(n) that a good hash function and load factor make very unlikely.
- Honest limit of the evidence: at this load, binary search alone would also be perfectly adequate, because the measured difference between the two is a matter of seconds per day. I chose the hash table because it has the lowest total daily cost and its search cost does not grow with n, while binary search stays available for range queries. If memory were tight or range queries were the main use, retaining the original binary-search design would also be a defensible choice.

In short, modify the design, with a hash table for exact lookups and the sorted array kept for ordered queries, rebuilt once a day.

Appendix – Code Used for the Experiments

This is the exact program that produced the numbers in Tasks 9, 10 and 12. Run it as it is with Python 3 to reproduce the comparison counts. Timings will vary slightly with the machine.

```
import random, time, sys
sys.setrecursionlimit(10000)


def linear(arr, key):                  # Approach A (iterative)
    c = 0
    for x in arr:
        c += 1
        if x == key: return True, c
    return False, c


def bs_rec(arr, key, lo, hi, cnt):         # Approach B (recursive)
    if lo > hi: return False
    mid = (lo + hi) // 2
    cnt[0] += 1
    if arr[mid] == key: return True
    if key < arr[mid]: return bs_rec(arr, key, lo, mid-1, cnt)
    return bs_rec(arr, key, mid+1, hi, cnt)
```

```
class HT:                           # Approach C (hash table, chaining)
    def __init__(self, n):
        self.m = max(8, int(n / 0.75)) | 1  # load factor ~0.75, odd size
        self.t = [None] * self.m
    def add(self, k):
        i = k % self.m
        if self.t[i] is None: self.t[i] = [k]
        else: self.t[i].append(k)
    def find(self, k):
        c = 0
        b = self.t[k % self.m]
        if b is None: return False, 0
        for x in b:
            c += 1
            if x == k: return True, c
        return False, c

random.seed(42)
Q = 1000
for n in [1000, 10000, 100000, 1000000]:
    data = random.sample(range(20200000, 20200000 + n*20), n)
    present = random.sample(data, Q // 2)
```

```
absent = [20200000 + n*20 + i for i in range(Q // 2)]
queries = present + absent
random.shuffle(queries)

t = time.perf_counter(); s = sorted(data)
sort_ms = (time.perf_counter() - t) * 1000

t = time.perf_counter(); h = HT(n)
for k in data: h.add(k)
build_ms = (time.perf_counter() - t) * 1000

lq = queries if n <= 100000 else queries[:100]
t = time.perf_counter(); lc = 0
for q in lq: lc += linear(data, q)[1]
lin_ms = (time.perf_counter() - t) * 1000 / len(lq)

t = time.perf_counter(); bc = 0
for q in queries:
    cnt = [0]; bs_rec(s, q, 0, n-1, cnt); bc += cnt[0]
bin_ms = (time.perf_counter() - t) * 1000 / Q

t = time.perf_counter(); hc = 0
for q in queries: hc += h.find(q)[1]
```

```
hash_ms = (time.perf_counter() - t) * 1000 / Q


S = set(data)                       # correctness check
for q in queries[:200]:
    cnt = [0]
    assert linear(data, q)[0] == bs_rec(s, q, 0, n-1, cnt) \
           == h.find(q)[0] == (q in S)


print(n, lin_ms, lc/len(lq), bin_ms, bc/Q, hash_ms, hc/Q, sort_ms, build_ms)
```